

PAPER

Improved Differential Fault Analysis of SOSEMANUK with Algebraic Techniques

Hao CHEN[†], Tao WANG[†], Shize GUO^{††}, Xinjie ZHAO^{††}, Fan ZHANG^{†††,††††a}, *Nonmembers*,
and Jian LIU^{††††}, *Student Member*

SUMMARY The differential fault analysis of SOSEMANUK was presented in Africacrypt in 2011. In this paper, we improve previous work with algebraic techniques which can result in a considerable reduction not only in the number of fault injections but also in time complexity. First, we propose an enhanced method to determine the fault position with a success rate up to 99% based on the single-word fault model. Then, instead of following the design of SOSEMANUK at word levels, we view SOSEMANUK at bit levels during the fault analysis and calculate most components of SOSEMANUK as bit-oriented. We show how to build algebraic equations for SOSEMANUK and how to represent the injected faults in bit-level. Finally, an SAT solver is exploited to solve the combined equations to recover the secret inner state. The results of simulations on a PC show that the full 384 bits initial inner state of SOSEMANUK can be recovered with only 15 fault injections in 3.97h.

key words: *eSTREAM, SOSEMANUK, algebraic attack, fault analysis, stream ciphers based on ARX operations*

1. Introduction

Fault attacks are a class of side-channel attacks where an attacker deduces the secret information by provoking a computational fault into the cryptosystem. The idea was originally introduced to attack CRT-RSA by Boneh et al. in 1996 [1]. Subsequently, the method was extended to attack the implementations of symmetric cryptographic algorithms such as DES by Biham and Shamir who combined it with differential cryptanalysis and proposed the so-called differential fault analysis [2]. In 2004, Hoch and Shamir showed that fault analysis can be applied to stream ciphers such as LILI-128, SOBER-t32, RC4 and those based on LFSRs [3]. After that, fault analysis has been successfully applied to break public cipher such as ECC [4], block ciphers such as AES [5], ARIA [7], and LED [16], and stream ciphers such as Trivium [10], MICKEY 2.0 [15] and MUGI [8]. Nowadays, fault

analysis is considered as one of the most threatening attacks for the cryptographic algorithms.

SOSEMANUK [20] is a fast synchronous software-oriented stream cipher. It was designed by Berbain et al. in 2005 as a submission to the eSTREAM [21] competition and was selected as a part of eSTREAM's final software portfolio. It uses a 128-bit initialization vector and has a key length varying from 128 to 256 bits, whereas the claimed security is always 128-bit. The design of SOSEMANUK is based on the stream cipher SNOW [22] and its transformations are derived from the block cipher SERPENT [23]. The motivation of the proposal of SOSEMANUK is to improve both the security and the efficiency of SNOW 2.0 [24].

1.1 Related Work

After the eSTREAM project is closed, several differential fault analyses against SOSEMANUK have been presented. The first differential fault analysis attack against SOSEMANUK was proposed by Yaser et al. in Africacrypt 2011 [18]. In this attack, a single-word fault model was adopted, where the adversary was assumed to be able to induce a random fault that alters the value of one random inner state register between the iterations of the cipher. The adversary has no control or knowledge over the exact location of faults. The authors first used faults occurred in the register s_4 and s_5 to restrict the input for each of the S-boxes. After that, several equations were exploited to provide restrictions on the LFSR registers. Finally, a guess-and-determine strategy was used to recover the rest of inner states. The time complexity of the whole attack was 2^{48} SOSEMANUK iterations and $2^{38.17}$ bytes memory space and 6144 fault injections were required as shown in Column 1 of Table 1.

The second differential fault analysis attack against SOSEMANUK was proposed by Ma et al. [19] which is listed in Column 2 of Table 1, which reduces the time complexity to $2^{35.16}$ SOSEMANUK iterations and the amount of required faults and memory space to 4608 and $2^{23.46}$ bytes, respectively. The major improvement from [18] is that the

Manuscript received May 26, 2016.

Manuscript revised October 21, 2016.

[†]The authors are with the Department of Information Engineering, Ordnance Engineering College, Shijiazhuang 050003, China.

^{††}The authors are with the Institute of North Electronic Equipment, Beijing 10083, China.

^{†††}The author is with the College of Information Science and Electrical Engineering, Zhejiang University, Hangzhou 310027, China.

^{††††}The author is also with Science and Technology on Communication Security Laboratory, Chengdu, China, 610041.

^{†††††}The author is with College of Communication and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China.

a) E-mail: fanzhang@zju.edu.cn

DOI: 10.1587/transfun.E100.A.811

Table 1 Comparisons of this paper with previous work.

Ref	[18]	[19]	This paper
Fault model	Single-word	Single-word	Single-word
Time Complexity	2^{48} iterations	$2^{35.16}$ iterations	3.97h
Memory requirement	$2^{38.17}$	$2^{23.46}$	$2^{21.10}$
#Fault numbers	6144	4608	15

candidates of input to each of S-boxes were further pruned by recognizing the type of faulty cipher instead of whether the faults are in register s_4 or s_5 .

1.2 Motivations

Previous fault analysis attacks against SOSEMANUK require a huge number of fault injections to recover the secret inner state of the cipher. Those aforementioned attacks meet two difficulties: one is to locate the exact registers where the faults were injected (except for faults occurred in the register s_4, s_5); the other is to restrict the S-box input-output values based on the experimental determination of S-box differences. It is possible to overcome these difficulties and reduce the number of required injections.

Note that SOSEMANUK is designed as word oriented (32 bits), however its underlying calculations are based on S-boxes and so-called ARX operations (addition modular 2^n , XOR and bit rotation). Actually both S-boxes and ARX operations can be calculated at bit levels. Based on this fact, both the cipher and the injected faults can be described as a series of algebraic equations in GF(2), resulting in a feasible algebraic fault analysis in practice. Since the experimental determination of S-box differences is not required, the required number of fault injections is significantly reduced.

The motivation of this paper is to pursue an enhanced attack to previous differential fault attacks. Three important issues need to be carefully discussed: how to determine the fault positions more accurately, how to build the algebraic equations for the cipher and injected faults, and how to conduct more efficient fault analysis against SOSEMANUK with algebraic techniques.

1.3 Contributions

The main contributions are summarized as follows.

- (1) An enhanced method is proposed to locate the fault positions. The propagation properties of the injected faults are carefully studied and exploited to effectively determining the position of faults. Experimental results show that the success rate of the method is more than 99.2%.
- (2) The algebraic description of the cipher is given. Some properties of SOSEMANUK and several algebraic techniques are provided to represent algebraic equations for all operations in the cipher from the view of bit units.
- (3) An efficient method is provided to minimize the number of fault injections. Every injected fault is represented as a series of simple algebraic equations according to its position information, and more low degree algebraic equations can be extracted for faults that were injected in $\{s_5, s_6, s_9\}$.
- (4) Our attack on SOSEMANUK is conducted in a common PC and an SAT solver is used to solve the combined equations of the cipher and faults automatically. Experimental results show that the attack can recover the secret inner states of the cipher in 3.97 hours with only 15 faults, which has made great improvement to the

previous differential fault analysis attacks against SOSEMANUK.

2. Preliminaries

2.1 Notations

In the rest of the paper, the following notations are used:

$\boxplus, \boxtimes$: $x \boxplus y$ means $x + y \bmod 2^{32}$, $x \boxtimes y$ means $x \times y \bmod 2^{32}$, where $0 \leq x < 2^{32}, 0 \leq y < 2^{32}$.

$\oplus, ||$: bitwise XOR and concatenation.

$\ll$: bitwise left rotation of a 32-bit value

' : sign for denoting faulty cipher register or output

$\#(\Gamma)$: number of elements in set Γ

$\Phi_0 = (R2_0, R1_0, s_{10}, s_9, s_8, s_7, s_6, s_5, s_4, s_3, s_2, s_1)$: the initial inner state of the stream cipher SOSEMANUK

x^i : the i -th bit of an n -bit word x is denoted by x^i

$X_i = f_{t+3}^i || f_{t+2}^i || f_{t+1}^i || f_t^i$ ($0 \leq i \leq 31$): the input value for i -th S-box applied in the Serpent1 function at time t .

2.2 Brief Description of SOSEMANUK

In this section, we briefly recall the SOSEMANUK algorithm. The full specification of SOSEMANUK can be found in [20]. As depicted in Fig. 1, SOSEMANUK consists of three main components: a linear feedback shift register (LFSR), a finite state machine (FSM) and a round function Serpent 1.

The LFSR of SOSEMANUK is defined over the finite field $\mathbb{F}_{2^{32}}$, and contains ten of 32-bit registers s_i , $1 \leq i \leq 10$. The feedback polynomial $\pi(X)$ of LFSR is defined as follows

$$\pi(X) = \alpha X^{10} + \alpha X^7 + X + 1 \in \mathbb{F}_{2^{32}}[X] \quad (1)$$

where α is a root of the polynomial

$$P(X) = X^4 + \beta^{23} X^3 + \beta^{245} X^2 + \beta^{48} X + \beta^{239} \quad (2)$$

on $\mathbb{F}_{2^8}[X]$, and β is a root of the binary polynomial

$$Q(X) = X^8 + X^7 + X^5 + X^3 + 1 \quad (3)$$

on $\mathbb{F}_2[X]$. The update function of LFSR is defined as follows:

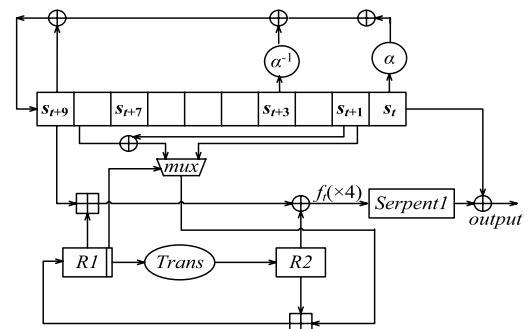


Fig. 1 The structure of SOSEMANUK.

$$s_{t+10} = s_{t+9} \oplus \alpha^{-1}s_{t+3} \oplus \alpha s_t, \forall t \geq 1 \quad (4)$$

The FSM is a nonlinear component with two 32-bit registers $R1$ and $R2$ described as the following relations:

$$R1_t = R2_{t-1} \boxplus (r_{t-1}s_{t+1} \oplus s_{t+8}) \quad (5)$$

$$R2_t = \text{Trans}(R1_{t-1}) \quad (6)$$

$$f_t = (s_{t+9} \boxplus R1_t) \oplus R2_t \quad (7)$$

where r_{t-1} denotes the least significant bit of $R1_{t-1}$. The internal function Trans is defined by Eq. (8) where M is the constant value $0x54655307$.

$$\text{Trans}(z) = (M \times z \bmod 2^{32})_{\ll 7} \quad (8)$$

The SOSEMANUK cipher combines the FSM and the LFSR to generate output values z_t . At the time $t = 0$, the internal state is initialized, and the first output value is z_1 . Each cipher iteration consists of four steps: first, the outputs of the FSM are grouped by four; then *Serpent1* is applied to each group; the result is XORed with the corresponding dropped values from LFSR to produce the output values. Once every four cipher iterations, four output values z_{t+3} , z_{t+2} , z_{t+1} , z_t are produced from f_t , f_{t+1} , f_{t+2} , f_{t+3} and s_{t+3} , s_{t+2} , s_{t+1} , s_t as follows

$$(z_{t+3}, z_{t+2}, z_{t+1}, z_t) = \text{Serpent1}(f_{t+3}, f_{t+2}, f_{t+1}, f_t) \oplus (s_{t+3}, s_{t+2}, s_{t+1}, s_t) \quad (9)$$

Serpent1 takes 32-bit word X_i as the input and outputs four 32 bits. It is defined by 32 applications of S in the bit-slice mode. S is the S-box S_2 of SERPENT.

$$S = [8, 6, 7, 9, 3, 12, 10, 15, 13, 1, 14, 4, 0, 11, 5, 2] \quad (10)$$

3. Some Properties of SOSEMANUK

In this section, we view one bit as a basic data unit and give some basic properties of SOSEMANUK from the view of bit units.

Define X as a 32-bit word in $\mathbb{F}_{2^{32}}$. Since $\mathbb{F}_{2^{32}}$ is defined as the quotient $\mathbb{F}_{2^8}[X]/P(X)$ and $\mathbb{F}_{2^8}$ is defined as the quotient $\mathbb{F}_2[X]/Q(X)$, we can denote X on the basis $(\beta^7, \beta^6, \dots, \beta, 1)$, $(\alpha^3, \alpha^2, \alpha^1, 1)$ as follows

$$\begin{aligned} X = & (x_{31}\beta^7 + x_{30}\beta^6 + \dots x_{24})\alpha^3 \\ & + (x_{23}\beta^7 + x_{22}\beta^6 + \dots x_{16})\alpha^2 \\ & + (x_{15}\beta^7 + x_{14}\beta^6 + \dots x_8)\alpha \\ & + (x_7\beta^7 + x_6\beta^6 + \dots x_0) \end{aligned} \quad (11)$$

If we view a byte as a basic data unit, X can be denote by

$$X = X^{(3)} \parallel X^{(2)} \parallel X^{(1)} \parallel X^{(0)} \quad (12)$$

If we view a bit as a basic data unit, X can be denote by

$$X = (x_{31}, x_{30}, \dots, x_0) \quad (13)$$

Lemma 1. Let s_t , s_{t+3} , αs_t , $\alpha^{-1}s_t$ be four 32-bit words $s_t = (x_{31}, x_{30}, \dots, x_0)$, $s_{t+3} = (y_{31}, y_{30}, \dots, y_0)$, $\alpha s_t = (z_{31}, z_{30}, \dots, z_0)$, $\alpha^{-1}s_t = (t_{31}, t_{30}, \dots, t_0)$ with x_0, y_0, z_0, t_0 being the least significant bit, respectively. Then αs_t , $\alpha^{-1}s_t$ can be written in the following bit-based representation:

Proof. Since β is the root of $Q(X)$, we have

$$\beta^8 = \beta^7 + \beta^5 + \beta^3 + 1 \quad (14)$$

In addition, according to [4], we have

$$\begin{aligned} \alpha s_t = & (\beta^{23}s_t^{(3)} + s_t^{(2)})\alpha^3 + (\beta^{245}s_t^{(3)} + s_t^{(1)})\alpha^2 \\ & + (\beta^{48}s_t^{(3)} + s_t^{(0)})\alpha + \beta^{239}s_t^{(3)} \end{aligned} \quad (15)$$

$$\begin{aligned} \alpha^{-1}s_{t+3} = & \beta^{16}s_{t+3}^{(0)}\alpha^3 + (s_{t+3}^{(3)} + \beta^{39}s_{t+3}^{(0)})\alpha^2 \\ & + (s_{t+3}^{(2)} + \beta^6s_{t+3}^{(0)})\alpha + (s_{t+3}^{(1)} + \beta^{64}s_{t+3}^{(0)}) \end{aligned} \quad (16)$$

Combining the two equations above and Eq. (14), we have

$$\begin{aligned} \alpha s_t = & [(\beta^7 + \beta^6 + \beta^5 + 1)s_t^{(3)} + s_t^{(2)}]\alpha^3 \\ & + [(\beta^7 + \beta^4 + \beta^3 + \beta^2 + \beta + 1)s_t^{(3)} + s_t^{(1)}]\alpha^2 \\ & + [(\beta^7 + \beta^6 + \beta^3 + \beta^2 + \beta + 1)s_t^{(3)} + s_t^{(0)}]\alpha \\ & + (\beta^4 + \beta + 1)s_t^{(3)} \\ \triangleq & \left(\sum_{i=0}^7 z_{24+i}\beta^i\right)\alpha^3 + \left(\sum_{i=0}^7 z_{16+i}\beta^i\right)\alpha^2 \\ & + \left(\sum_{i=0}^7 z_{8+i}\beta^i\right)\alpha + \sum_{i=0}^7 z_i\beta^i \end{aligned}$$

Thus:

$$\begin{aligned} & \left(\sum_{i=0}^7 z_{24+i}\beta^i\right)\alpha^3 \\ = & \left[\begin{aligned} & (x_{23} + x_{24} + x_{26})\beta^7 \\ & + (x_{22} + x_{24} + x_{25} + x_{26} + x_{31})\beta^6 \\ & + (x_{21} + x_{24} + x_{25} + x_{30})\beta^5 \\ & + (x_{20} + x_{26} + x_{29} + x_{31})\beta^4 \\ & + (x_{19} + x_{25} + x_{28} + x_{30})\beta^3 \\ & + (x_{18} + x_{26} + x_{27} + x_{29})\beta^2 \\ & + (x_{17} + x_{25} + x_{26} + x_{28})\beta \\ & + x_{16} + x_{24} + x_{25} + x_{27} \end{aligned} \right] \alpha^3 \end{aligned}$$

Therefore, we have

$$\begin{cases} z_{31} = x_{23} \oplus x_{24} \oplus x_{26} \\ z_{30} = x_{22} \oplus x_{24} \oplus x_{25} \oplus x_{26} \oplus x_{31} \\ z_{29} = x_{21} \oplus x_{24} \oplus x_{25} \oplus x_{30} \\ z_{28} = x_{20} \oplus x_{26} \oplus x_{29} \oplus x_{31} \\ z_{27} = x_{19} \oplus x_{25} \oplus x_{28} \oplus x_{30} \\ z_{26} = x_{18} \oplus x_{26} \oplus x_{27} \oplus x_{29} \\ z_{25} = x_{17} \oplus x_{25} \oplus x_{26} \oplus x_{28} \\ z_{24} = x_{16} \oplus x_{24} \oplus x_{25} \oplus x_{27} \end{cases}$$

Table 2 The expression of αs_t , $\alpha^{-1} s_t$ in bit form.

αs_t	$\alpha^{-1} s_t$
$z_0 = x_{24} + x_{28} + x_{29} + x_{30} + x_{31}$	$t_0 = y_0 + y_1 + y_4 + y_7 + y_8$
$z_1 = x_{24} + x_{25} + x_{29} + x_{30} + x_{31}$	$t_1 = y_1 + y_2 + y_5 + y_9$
$z_2 = x_{25} + x_{26} + x_{30} + x_{31}$	$t_2 = y_0 + y_2 + y_3 + y_6 + y_{10}$
$z_3 = x_{26} + x_{27} + x_{28} + x_{29} + x_{30}$	$t_3 = y_0 + y_3 + y_{11}$
$z_4 = x_{24} + x_{27} + x_{28}$ $+ x_{29} + x_{30} + x_{31}$	$t_4 = y_1 + y_4 + y_{12}$
$z_5 = x_{25}$	$t_5 = y_1 + y_2 + y_4 + y_5 + y_7 + y_{13}$
$z_6 = x_{26}$	$t_6 = y_0 + y_2 + y_3 + y_5 + y_6 + y_{14}$
$z_7 = x_{27} + x_{28} + x_{29} + x_{30} + x_{31}$	$t_7 = y_0 + y_3 + y_6 + y_{15}$
$z_8 = x_0 + x_{24} + x_{25} + x_{28}$	$t_8 = y_2 + y_3 + y_4 + y_6 + y_7 + y_{16}$
$z_9 = x_1 + x_{24} + x_{25} + x_{26} + x_{29}$	$t_9 = y_3 + y_4 + y_5 + y_7 + y_{17}$
$z_{10} = x_2 + x_{24} + x_{25}$ $+ x_{26} + x_{27} + x_{30}$	$t_{10} = y_4 + y_5 + y_6 + y_{18}$
$z_{11} = x_3 + x_{24} + x_{26} + x_{27} + x_{31}$	$t_{11} = y_2 + y_3 + y_4 + y_5 + y_{19}$
$z_{12} = x_4 + x_{25} + x_{27} + x_{28}$	$t_{12} = y_3 + y_4 + y_5 + y_6 + y_{20}$
$z_{13} = x_5 + x_{25} + x_{26} + x_{29}$	$t_{13} = y_2 + y_3 + y_5 + y_{21}$
$z_{14} = x_6 + x_{24} + x_{26} + x_{27} + x_{30}$	$t_{14} = y_0 + y_3 + y_4 + y_6 + y_{22}$
$z_{15} = x_7 + x_{24} + x_{27} + x_{31}$	$t_{15} = y_1 + y_2 + y_3 + y_5 + y_6 + y_{23}$
$z_{16} = x_8 + x_{24} + x_{25} + x_{26}$ $+ x_{27} + x_{28} + x_{29} + x_{31}$	$t_{16} = y_0 + y_5 + y_7 + y_{24}$
$z_{17} = x_9 + x_{24} + x_{25} + x_{26}$ $+ x_{27} + x_{28} + x_{29} + x_{30}$	$t_{17} = y_0 + y_1 + y_6 + y_{25}$
$z_{18} = x_{10} + x_{24} + x_{25} + x_{26} + x_{27}$ $+ x_{28} + x_{29} + x_{30} + x_{31}$	$t_{18} = y_0 + y_1 + y_2 + y_7 + y_{26}$
$z_{19} = x_{11} + x_{24} + x_{30}$	$t_{19} = y_0 + y_1 + y_2 + y_3$ $+ y_5 + y_7 + y_{27}$
$z_{20} = x_{12} + x_{24} + x_{25} + x_{31}$	$t_{20} = y_1 + y_2 + y_3 + y_4 + y_6 + y_{28}$
$z_{21} = x_{13} + x_{27} + x_{28} + x_{29} + x_{31}$	$t_{21} = y_2 + y_3 + y_4 + y_{29}$
$z_{22} = x_{14} + x_{28} + x_{29} + x_{30}$	$t_{22} = y_3 + y_4 + y_5 + y_{30}$
$z_{23} = x_{15} + x_{24} + x_{25} + x_{26}$ $+ x_{27} + x_{28} + x_{30}$	$t_{23} = y_0 + y_1 + y_5$
$z_{24} = x_{16} + x_{24} + x_{25} + x_{27}$	$t_{24} = y_1 + y_2 + y_4 + y_6 + y_7$
$z_{25} = x_{17} + x_{25} + x_{26} + x_{28}$	$t_{25} = y_5$
$z_{26} = x_{18} + x_{26} + x_{27} + x_{29}$	$t_{26} = y_6$
$z_{27} = x_{19} + x_{25} + x_{28} + x_{30}$	$t_{27} = y_0 + y_4$
$z_{28} = x_{20} + x_{26} + x_{29} + x_{31}$	$t_{28} = y_0 + y_1 + y_5$
$z_{29} = x_{21} + x_{24} + x_{25} + x_{30}$	$t_{29} = y_1 + y_2 + y_4 + y_6 + y_7$
$z_{30} = x_{22} + x_{24} + x_{25} + x_{26} + x_{31}$	$t_{30} = y_2 + y_3 + y_5 + y_7$
$z_{31} = x_{23} + x_{24} + x_{26}$	$t_{31} = y_3 + y_6 + y_7$

In the similar way, we can get the desired conclusions as shown in Table 2. $\square$

4. Attack Method

4.1 Fault Model

In this paper, we adopt the same fault model as in [18], [19]. Namely, the adversary is allowed to inject one transient single-word fault at a random position within the inner states between the iterations, but he cannot control the exact position of the fault and he does not know the exact value of the fault either. The adversary is also assumed to be able to re-initialize the cipher for an arbitrary number of times with the same secret key and IV.

4.2 The Determination of the Fault Position

Since the adversary does not know the exact position in the assumed fault model, the first step of our attack is to determine the fault position. The idea is that we can find

out the position by observing the number of non-activated S-boxes during the first and second four iterations.

Let us consider the case where a fault has been injected to s_{t+5} right after the SOSEMANUK initialization step as described in Fig. 2. The fault moves to the right as the LFSR is clocked.

Obviously, no faulty values are involved in the generation process of f_t . Furthermore, in every step the FSM is first updated, then f_t is computed and LFSR is clocked. Therefore f_{t+1} and f_{t+2} are computed without errors and the fault only affects f_{t+3} . During the next four cipher iterations, the fault affects f_{t+4} , f_{t+5} , f_{t+6} , f_{t+7} . When the fault occurs at other registers, the fault changes f_t , f_{t+1} , f_{t+2} , f_{t+3} , f_{t+4} , f_{t+5} , f_{t+6} , f_{t+7} in the similar way, which is as shown in Table 3 (“ $\checkmark$ ” and “ $\times$ ” mean with or without corruptions).

For the clarifications, we use the variable pos to denote the register in which the fault is injected. Δ_1 and Δ_2 stand for the input differences of the 32 S-boxes of *Serpent*1 during the first and second four consecutive cipher iterations, respectively. Following the explanation given above, for simplicity, we write Δ_1 and Δ_2 in hexadecimals. For example, if the input differences of the 32 S-boxes of *Serpent*1 during the first four cipher iterations are 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9, 0xA, 0xB, 0xC, 0xD, 0xE, 0xF, $\Delta_1 = 0x1111111111111111 = 0xFFFE$. We can group the fault positions in Table 3 into 7 categories according to the value of pos :

C₁: $pos \in \{s_2, s_{10}, s_9 \text{ and } lsb(R_{10}) = 1, R_{10}, R_{20}\}$, $\Delta_1 = \Delta_2 = 0xFFFE$

C₂: $pos \in \{s_1, s_3, s_4\}$, $\Delta_1 = 0x6554$, $\Delta_2 = 0xFFFE$

C₃: $pos \in \{s_5\}$, $\Delta_1 = 0x1110$, $\Delta_2 = 0xFFFE$

C₄: $pos \in \{s_6\}$, $\Delta_1 = 0x0100$, $\Delta_2 = 0xFFFE$

C₅: $pos \in \{s_7\}$, $\Delta_1 = 0x0000$, $\Delta_2 = 0xFFFE$

C₆: $pos \in \{s_8\}$, $\Delta_1 = 0x0000$, $\Delta_2 = 0x6554$

C₇: $pos \in \{s_9 \text{ and } lsb(R_{10}) = 0\}$, $\Delta_1 = 0x0000$, $\Delta_2 = 0x1110$

Lemma 2. Let κ denote the set of candidates for fault positions. m and n denote the number of non-activated S-boxes in the application of the 32 S-boxes of *Serpent* 1 during the first and second four iterations respectively.

$$m = \# \{z_4^i | z_3^i | z_2^i | z_1^i = z_4^i | z_3^i | z_2^i | z_1^i, 0 \leq i \leq 31\}$$

$$n = \# \{z_8^i | z_7^i | z_6^i | z_5^i = z_8^i | z_7^i | z_6^i | z_5^i, 0 \leq i \leq 31\}$$

We have the following conclusions:

1. If $0 \leq m \leq 1$, the fault most likely (with the probability approximating to 1, the same below) belongs to C_1 and C_2 , $\kappa = \{s_1, s_2, s_3, s_4, s_9, s_{10}, R_{10}, R_{20}\}$
2. If $2 \leq m \leq 6$, the fault most likely belongs to C_1 , C_2 , C_3 , $\kappa = \{s_1, s_2, s_3, s_4, s_5, s_9, s_{10}, R_{10}, R_{20}\}$
3. If $7 \leq m \leq 9$, the fault most likely belongs to C_1 , C_2 , C_3 , C_4 , $\kappa = \{s_1, s_2, s_3, s_4, s_5, s_6, s_9, s_{10}, R_{10}, R_{20}\}$
4. If $10 \leq m \leq 13$, the fault most likely belongs to C_1 , C_3 , C_4 , $\kappa = \{s_1, s_3, s_4, s_5, s_6\}$
5. If $14 \leq m \leq 18$, the fault most likely belongs to C_3 , C_4 , $\kappa = \{s_5, s_6\}$
6. If $19 \leq m \leq 31$, the fault most likely belongs to C_4 ,

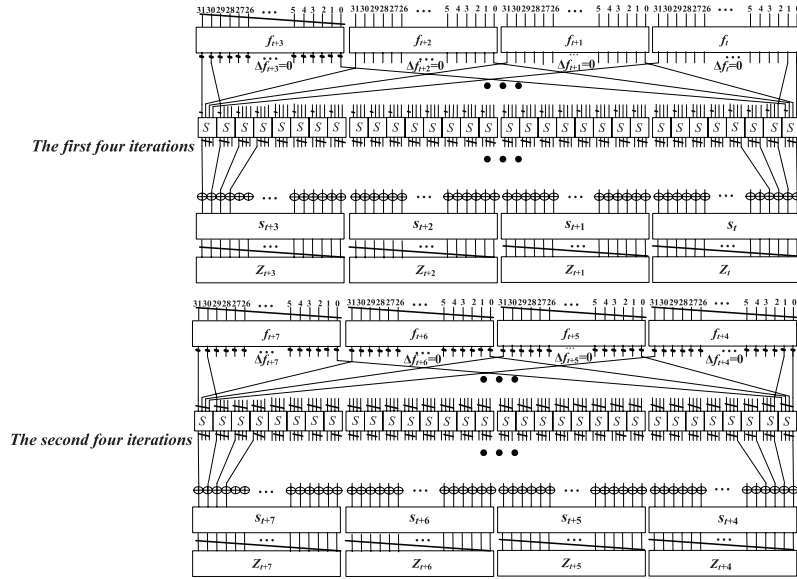


Fig. 2 The propagation of the injected fault.

Table 3 Fault type.

Fault position	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
s_2, s_9 ($lsb(R1_0) = 1$), $s_{10}, R1_0, R2_0$	×	×	×	×	×	×	×	×
s_1, s_3, s_4	✓	×	×	×	×	×	×	×
s_5	✓	✓	×	×	×	×	×	×
s_6	✓	✓	✓	×	×	×	×	×
s_7	✓	✓	✓	✓	×	×	×	×
s_8	✓	✓	✓	✓	✓	×	×	×
s_9 ($lsb(R1_0) = 0$)	✓	✓	✓	✓	✓	✓	×	×

$\kappa = \{s_6\}$

7. If $m = 32$ and $0 \leq n \leq 9$ the fault most likely belongs to C_5, C_6, C_7 , $\kappa = \{s_7, s_8, s_9\}$
8. If $m = 32$ and $10 \leq n \leq 15$ the fault the fault most likely belongs to C_5, C_6 , $\kappa = \{s_7, s_8\}$
9. If $m = 32$ and $n \geq 16$ the fault most likely belongs to C_7 , $\kappa = \{s_9\}$

Proof. Without the loss of generality, we only prove Case (1). The proofs for other cases are listed in Appendix 1.

Let D_k ($1 \leq k \leq 7$), be the event that a fault belongs to C_k under the random fault model. Since the faults are induced completely random, $P[D_1] = \frac{9}{24}$. And according to Table 3, the probability that a particular S-box will not be activated given that the fault has occurred at s_2, s_9 ($lsb(R1_0) = 0$), $s_{10}, R1_0, R2_0$ is 2^{-4} . Thus, it is obviously that $m \sim B(2^{-4}, 32)$, $n \sim B(2^{-4}, 32)$. More generally, the value of $P[D_k]$ in terms of the fault position is given as follows:

1. $P[D_1] = \frac{9}{24}$, $m \sim B(2^{-4}, 32)$, $n \sim B(2^{-4}, 32)$
2. $P[D_2] = \frac{1}{4}$, $m \sim B(2^{-3}, 32)$, $n \sim B(2^{-4}, 32)$
3. $P[D_3] = \frac{1}{12}$, $m \sim B(2^{-2}, 32)$, $n \sim B(2^{-4}, 32)$

$$4. P[D_4] = \frac{1}{12}, m \sim B(2^{-1}, 32), n \sim B(2^{-4}, 32)$$

$$5. P[D_5] = \frac{1}{12}, P[m = 32] = 1, P[n < 32] = 0, n \sim B(2^{-4}, 32)$$

$$6. P[D_6] = \frac{1}{12}, P[m = 32] = 1, P[n < 32] = 0, n \sim B(2^{-3}, 32)$$

$$7. P[D_7] = \frac{1}{24}, P[m = 32] = 1, P[n < 32] = 0, n \sim B(2^{-2}, 32)$$

For Case (1), let A be the event that the injected faults belong to C_1 or C_2 , B_1 be the event that $m = 0$, B_2 be the event that $m = 1$. Then, we can apply the Bayesian formula to derive the conditional possibility $P[A|B_1]$ by

$$P[A|B_1] = \frac{P[AB_1]}{P[B_1]} \quad (17)$$

where

$$P[B_1] = \sum_{k=1}^7 (1 - P[E_k])^{32} P[D_k] \quad (18)$$

$$P[AB_1] = \sum_{k=1}^2 (1 - P[E_k])^{32} P[D_k] \quad (19)$$

where $P[E_k]$ denotes the probability that a particular S-boxes will not be activated given that the fault has occurred at C_k . Thus, we have $P[A|B_1] = 0.9999$. In the similar way, we can get $P[A|B_2] = 0.9992$. Therefore, for the case of $0 \leq m \leq 1$, the fault should belong to C_1, C_2 and $\kappa = \{s_1, s_2, s_3, s_4, s_9, s_{10}, R1_0, R2_0\}$. $\square$

4.3 Further Pruning of the Fault Positions Candidates

Lemma 3. Let $\Upsilon_{(i,k)}$ denote the values of output differences

of S-boxes for the fault belongs to C_i , $1 \leq i \leq 7$, during the k -th four cipher iterations, then we have the following statements.

- (1) $\Upsilon_{(1,1)} = \Upsilon_{(3,1)}$
- (2) $\Upsilon_{(2,1)} - \Upsilon_{(1,1)} = 0x1002$
- (3) $\Upsilon_{(1,1)} - \Upsilon_{(4,1)} = 0x4D54$
- (4) $\Upsilon_{(2,1)} - \Upsilon_{(4,1)} = 0x4D56$
- (5) $\Upsilon_{(5,2)} - \Upsilon_{(6,2)} = 0x1002$
- (6) $\Upsilon_{(5,2)} - \Upsilon_{(7,2)} = 0x1002$

Proof. According to Eq. (10) and Table 3, we have $\Upsilon_{(1,1)} = 0xEFFC$, $\Upsilon_{(2,1)} = 0xFFFE$, $\Upsilon_{(3,1)} = 0xEFFC$, $\Upsilon_{(4,1)} = 0xA2A8$, $\Upsilon_{(5,2)} = 0xFFFE$, $\Upsilon_{(6,2)} = 0xEFFC$, $\Upsilon_{(7,2)} = 0xEFFC$. With these identities, the desired statements can be deduced immediately. $\square$

Lemma 4. Assume a fault has been injected in:

- (1) $\{s_5, s_6, R1_0, R2_0\}$, then $\Upsilon_{(i,1)} = \Delta z$, otherwise $\Upsilon_{(i,1)} \subseteq \Delta z$
- (2) $\{s_7, s_8, s_9\}$, then $\Upsilon_{(i,2)} = \Delta z$, otherwise $\Upsilon_{(i,2)} \subseteq \Delta z$

Proof. It's obvious that a fault does not corrupt any value of $s_{t+3}, s_{t+2}, s_{t+1}, s_t$ during the first four cipher iterations when the fault was injected to $\{s_5, s_6, R1_0, R2_0\}$. In such a situation, $\Delta(z_{t+3}, z_{t+2}, z_{t+1}, z_t) = \Upsilon_{(i,1)}$ due to Eq. (7) by which no corrupted bit is XOR-ed with any bit of S-box. Similarly, $\Upsilon_{(i,2)} = \Delta z$ for faults injected in $\{s_7, s_8, s_9\}$. $\square$

Lemma 3 and 4 show that the output differences of the S-boxes can be calculated as the differences between the correct and faulty keystream directly for faults injected in $\{s_5, s_6, R1_0, R2_0\}$ and $\{s_7, s_8, s_9\}$. Once $\Upsilon_{(i,k)}$ was known, the number of candidates for fault positions can be further reduced as follows:

1. For $2 \leq m \leq 6$, if $\exists \beta \in \Delta_1$, $\beta \in \{0x1, 0xC\}$, the set κ can be further reduced to $\{s_1, s_2, s_3, s_4, s_9, s_{10}, R1_0, R2_0\}$.
2. For $7 \leq m \leq 9$, if $\exists \beta \in \Delta_1$, $\beta \in \{0x1, 0xC\}$, the set κ can be further reduced to $\{s_1, s_2, s_3, s_9, s_{10}, R1_0, R2_0\}$. If $\forall \beta \in \Delta_1$, $\beta \notin \{0x1, 0xC\}$, and $\exists \alpha \in \Delta_1$, $\alpha \in \{0x2, 0x4, 0x6, 0x8, 0xA, 0xB, 0xE\}$, $\kappa = \{s_1, s_2, s_3, s_4, s_9, s_{10}, R1_0, R2_0\}$.
3. For $10 \leq m \leq 13$, if $\exists \beta \in \Delta_1$, $\beta \in \{0x1, 0xC\}$, then the fault should be injected in $\{s_1, s_3, s_4\}$. If $\forall \beta \in \Delta_1$, $\beta \notin \{0x1, 0xC\}$ and $\exists \alpha \in \Delta_1$, $\alpha \in \{0x2, 0x4, 0x6, 0x8, 0xA, 0xB, 0xE\}$, the set κ can be further reduced to $\{s_1, s_3, s_4, s_5\}$.
4. For $14 \leq m \leq 18$, if $\exists \beta \in \Delta_1$, $\beta \in \Upsilon_{(3,1)} - \Upsilon_{(4,1)}$, then the fault should be injected in $\{s_5\}$.
5. For $m = 32$ and $1 \leq n \leq 9$, if $\exists \beta \in \Delta_2$, and $\beta \in \{0x1, 0xC\}$, then the fault should be injected in $\{s_7, s_8\}$.
6. For $m = 32$ and $10 \leq n \leq 15$, if $\exists \beta \in \Delta_2$, and $\beta \in \{0x1, 0xC\}$, then the fault should be injected in $\{s_8\}$.

4.4 Algebraic Description of SOSEMANUK on GF(2)

This section describes the algebraic representation of SOSEMANUK on GF(2).

Equations for *Serpent1*. The function *Serpent1* is defined as the application of 32 S-boxes in bitslice mode. Assume the input and output of one S-box are $X = (f_{t+3}, f_{t+2}, f_{t+1}, f_t)$, $Y = (y_{t+3}, y_{t+2}, y_{t+1}, y_t)$ where f_t and y_t are the least significant bit of X, Y respectively. The S-box in SOSEMANUK can be represented with the following equation.

$$\begin{cases} y_t = f_{t+1} \oplus f_{t+2} \oplus f_{t+3} \oplus f_t f_{t+2} \\ y_{t+1} = f_t \oplus f_{t+1} \oplus f_{t+2} \oplus f_t f_{t+3} \oplus f_{t+1} f_{t+2} \\ \oplus f_{t+2} f_{t+3} \oplus f_t f_{t+1} f_{t+2} \oplus f_t f_{t+1} f_{t+3} \oplus f_t f_{t+2} f_{t+3} \\ y_{t+2} = f_t \oplus f_{t+1} \oplus f_{t+3} \oplus f_{t+1} f_{t+2} \oplus f_{t+1} f_{t+3} \\ \oplus f_{t+2} f_{t+3} \oplus f_t f_{t+1} f_{t+3} \oplus f_t f_{t+2} f_{t+3} \\ y_{t+3} = f_t \oplus f_{t+1} \oplus f_{t+2} \oplus f_{t+1} f_{t+3} \oplus f_t f_{t+1} f_{t+2} \oplus 1 \end{cases} \quad (20)$$

Equations for addition modulo 2^n . Assume $X, Y, Z \in \text{GF}(2^n)$ are the two inputs and output of addition modulo 2^n where $X = (x_{n-1}, x_{n-2}, \dots, x_0)$, $Y = (y_{n-1}, y_{n-2}, \dots, y_0)$, $Z = (z_{n-1}, z_{n-2}, \dots, z_0)$, respectively. Then modulo 2^n addition can be represented in the bit form as follows

$$\begin{cases} z_0 = x_0 \oplus y_0 \\ z_1 = x_1 \oplus y_1 \oplus x_0 y_0 \\ z_2 = x_2 \oplus y_2 \oplus x_1 y_1 \oplus (x_1 \oplus y_1)(x_1 \oplus y_1 \oplus z_1) \\ \vdots \\ z_i = x_i \oplus y_i \oplus x_{i-1} y_{i-1} \oplus (x_{i-1} \oplus y_{i-1})(x_{i-1} \oplus y_{i-1} \oplus z_{i-1}) \\ \vdots \\ z_{n-1} = x_{n-1} \oplus y_{n-1} \oplus x_{n-2} y_{n-2} \oplus (x_{n-2} \oplus y_{n-2})(x_{n-2} \oplus y_{n-2} \oplus z_{n-2}) \end{cases} \quad (21)$$

Equations for the update function of LFSR. Let $s_{t+9} = (\omega_{31}, \omega_{30}, \dots, \omega_0)$, $s_{t+10} = (\lambda_{31}, \lambda_{30}, \dots, \lambda_0)$. According to the property of SOSEMANUK in Sect. 1.1, we get the bit representation of s_{t+10} as shown in Table 4.

Equations for *Trans(z)*. The equations for *Trans(z)* can be represented by the following equation where $M = 0x54655307$, and $\Lambda = \{30, 28, 26, 22, 21, 18, 16, 14, 12, 9, 8, 2, 1, 0\}$.

$$\begin{aligned} \text{Trans}(z) &= (M \times z)_{\ll 7} \\ &= (0x54655307 \times z)_{\ll 7} \\ &= ((01010100011001010101001100000111)_2 \times z)_{\ll 7} \\ &= \left(\bigoplus_{i \in \Lambda} (z)_{\ll i} \right)_{\ll 7} \end{aligned} \quad (22)$$

4.5 Algebraic Description of Faults on GF(2)

Each single-word fault injection can be represented with additional equations, which can be appended to the overall equation system to enhance the solving. Suppose a fault α

Table 4 The expression of s_{t+10} in bit form.

index	s_{t+10}
λ_0	$\omega_0 + x_{24} + x_{28} + x_{29} + x_{30} + x_{31} + y_0 + y_1 + y_4 + y_7 + y_8$
λ_1	$\omega_1 + x_{24} + x_{25} + x_{29} + x_{30} + x_{31} + y_1 + y_2 + y_5 + y_9$
λ_2	$\omega_2 + x_{25} + x_{26} + x_{30} + x_{31} + y_0 + y_2 + y_3 + y_6 + y_{10}$
λ_3	$\omega_3 + x_{26} + x_{27} + x_{28} + x_{29} + x_{30} + y_0 + y_3 + y_{11}$
λ_4	$\omega_4 + x_{24} + x_{27} + x_{28} + x_{29} + x_{30} + x_{31} + y_1 + y_4 + y_{12}$
λ_5	$\omega_5 + x_{25} + y_1 + y_2 + y_4 + y_5 + y_7 + y_{13}$
λ_6	$\omega_6 + x_{26} + y_0 + y_2 + y_3 + y_5 + y_6 + y_{14}$
λ_7	$\omega_7 + x_{27} + x_{28} + x_{29} + x_{30} + x_{31} + y_0 + y_3 + y_6 + y_{15}$
λ_8	$\omega_8 + x_0 + x_{24} + x_{25} + x_{28} + y_2 + y_3 + y_4 + y_6 + y_7 + y_{16}$
λ_9	$\omega_9 + x_1 + x_{24} + x_{25} + x_{26} + x_{29} + y_3 + y_4 + y_5 + y_7 + y_{17}$
λ_{10}	$\omega_{10} + x_2 + x_{24} + x_{25} + x_{26} + x_{27} + x_{30} + y_4 + y_5 + y_6 + y_{18}$
λ_{11}	$\omega_{11} + x_3 + x_{24} + x_{26} + x_{27} + x_{31} + y_2 + y_3 + y_4 + y_5 + y_{19}$
λ_{12}	$\omega_{12} + x_4 + x_{25} + x_{27} + x_{28} + y_3 + y_4 + y_5 + y_6 + y_{20}$
λ_{13}	$\omega_{13} + x_5 + x_{25} + x_{26} + x_{29} + y_2 + y_3 + y_5 + y_{21}$
λ_{14}	$\omega_{14} + x_6 + x_{24} + x_{26} + x_{27} + x_{30} + y_0 + y_3 + y_4 + y_6 + y_{22}$
λ_{15}	$\omega_{15} + x_7 + x_{24} + x_{27} + x_{31} + y_1 + y_2 + y_3 + y_5 + y_6 + y_{23}$
λ_{16}	$\omega_{16} + x_8 + x_{24} + x_{25} + x_{26} + x_{27} + x_{28} + x_{29} + x_{31}$ $+ y_0 + y_5 + y_7 + y_{24}$
λ_{17}	$\omega_{17} + x_9 + x_{24} + x_{25} + x_{26} + x_{27} + x_{28} + x_{29} + x_{30}$ $+ y_0 + y_1 + y_6 + y_{25}$
λ_{18}	$\omega_{18} + x_{10} + x_{24} + x_{25} + x_{26} + x_{27} + x_{28} + x_{29} + x_{30}$ $+ x_{31} + y_0 + y_1 + y_2 + y_7 + y_{26}$
λ_{19}	$\omega_{19} + x_{11} + x_{24} + x_{30} + y_0 + y_1 + y_2 + y_3 + y_5 + y_7 + y_{27}$
λ_{20}	$\omega_{20} + x_{12} + x_{24} + x_{25} + x_{31} + y_1 + y_2 + y_3 + y_4 + y_6 + y_{28}$
λ_{21}	$\omega_{21} + x_{13} + x_{27} + x_{28} + x_{29} + x_{31} + y_2 + y_3 + y_4 + y_{29}$
λ_{22}	$\omega_{22} + x_{14} + x_{28} + x_{29} + x_{30} + y_3 + y_4 + y_5 + y_{30}$
λ_{23}	$\omega_{23} + x_{15} + x_{24} + x_{25} + x_{26} + x_{27} + x_{28} + x_{30} + y_4$ $+ y_6 + y_7 + y_{31}$
λ_{24}	$\omega_{24} + x_{16} + x_{24} + x_{25} + x_{27} + y_4 + y_7$
λ_{25}	$\omega_{25} + x_{17} + x_{25} + x_{26} + x_{28} + y_5$
λ_{26}	$\omega_{26} + x_{18} + x_{26} + x_{27} + x_{29} + y_6$
λ_{27}	$\omega_{27} + x_{19} + x_{25} + x_{28} + x_{30} + y_0 + y_4$
λ_{28}	$\omega_{28} + x_{20} + x_{26} + x_{29} + x_{31} + y_0 + y_1 + y_5$
λ_{29}	$\omega_{29} + x_{21} + x_{24} + x_{25} + x_{30} + y_1 + y_2 + y_4 + y_6 + y_7$
λ_{30}	$\omega_{30} + x_{22} + x_{24} + x_{25} + x_{26} + x_{31} + y_2 + y_3 + y_5 + y_7$
λ_{31}	$\omega_{31} + x_{23} + x_{24} + x_{26} + y_3 + y_6 + y_7$

has been randomly injected in one of the 12 registers. The candidate register(s) in which the fault is most likely injected is(are) denoted by Σ . Then the fault α can be represented by the following equation:

$$\bigoplus_{\tau \in \Sigma} (\tau^i \oplus \tau'^i) \oplus \alpha^i = 0, \quad 0 \leq i \leq 31 \quad (23)$$

where τ^i and τ'^i denote the i -th bit of the element that is in Σ before and after the fault was injected, respectively. The registers that have not been corrupted by the fault are denoted by $\Phi_0 - \Sigma$, and for any element v in $\Phi_0 - \Sigma$

$$v^i \oplus v'^i = 0, \quad 0 \leq i \leq 31 \quad (24)$$

As mentioned in Sect. 4.3, once a fault has been injected in $\{s_5, s_6\}$ or $\{s_9\}$, we have $\Upsilon_{(i,1)} = \Delta z$ and $\Upsilon_{(i,2)} = \Delta z$, respectively. For any element $\beta \in \Upsilon_{(i,1)}$ or $\Upsilon_{(i,2)}$ the corresponding possible inputs to S-box can be denoted by

$$\Gamma_\beta = \{X'_i | S(X_i) \oplus S(X'_i) = \beta, 0 \leq i \leq 31\} \quad (25)$$

Then the k -th bit of the correct input $X_i = f_4^i | f_3^i | f_2^i | f_1^i$ to the i -th S-box at time $t = 1$ can be represented as follows:

$$f_k^i = \prod_{m=0}^{\#(\Gamma_\beta)-1} (f_k^m)^i,$$

Table 5 The values of x_i based on β, Γ_β .

β	Γ_β	X_i
0x2	0x2, 0xE	$f_1^i = 1, f_0^i = 0$
0x3	0x4, 0xC	$f_2^i = 1, f_1^i = 0, f_0^i = 0$
0x4	0x6, 0xA	$f_1^i = 1, f_0^i = 0$
0x5	0x0, 0x8	$f_2^i = 0, f_1^i = 0, f_0^i = 0$
0x6	0x3, 0x7, 0xB, 0xF	$f_1^i = 1, f_0^i = 1$
0x7	0x1, 0x5, 0x9, 0xD	$f_1^i = 0, f_0^i = 1$
0x8	0x0, 0xC	$f_1^i = 0, f_0^i = 0$
0x9	0x2, 0xA	$f_2^i = 0, f_1^i = 1, f_0^i = 0$
0xA	0x1, 0x5, 0x9, 0xD	$f_1^i = 0, f_0^i = 1$
0xE	0x4, 0x8, 0x2	$f_1^i = 0, f_0^i = 0$
0xF	0x6, 0xE	$f_2^i = 1, f_1^i = 1, f_0^i = 0$

$$\left((f_k^i)_{\#(\Gamma_\beta)-1} \cdots (f_k^i)_0, 1 \leq k \leq 4 \right) \quad (26)$$

To explain the construction of additional equations produced after injecting a fault clearly, we take the case that a fault has been injected to s_5 at time $t = 1$ as an example. In this case, the input differences of the S-boxes during the first four iterations are $\Delta_1 = 0x1110$ and the corresponding output differences of S-boxes are $\Upsilon_{(5,1)} = 0xEFFC$. Then, we can observe the following results as shown in Table 5.

Solving the combined equation system for Φ_0 . In this section, the initial inner state of the stream cipher SOSEMANUK Φ_0 can be recovered by solving the equation system of the cipher and faults with a solver.

There are many solvers for solving such a system, such as mutantXL algorithm, and Gröbner basis-based. However, one major problem for all those solvers is the memory usage for large sparse systems. Recently, SAT-based solvers have made a great progress. Therefore we have chosen SAT-based solvers for algebraic cryptanalysis, which is widely studied in previous work [6], [13], [14], [16], [17]. More specially, we choose the CryptoMiniSat v4.4 as the solver in our analysis which is a DPLL-based SAT solver developed from Minisat and has won the Gold Prizes in the SAT Race competition in 2010. The readers can refer to [9] for details of how to generate equations and how to feed them to the solver.

5. Attack Description

The aim of our attack is to recover the secret initial inner state of the SOSEMANUK. The adversary is assumed to inject k single-word faults randomly at time $t = 1$ with the following information:

- (1) The vector of the output keystream bits before the fault injection, $Z = z_1 | z_2 | z_3 | z_4 | z_5 | z_6 | z_7 | z_8$.
- (2) The vector of the output keystream bits that are obtained after each fault σ_i injection, $(Z^{(1)} \dots, Z^{(k)})$.

Algorithm 1 explains the procedure of the attack. The process of determining fault positions based on the number of non-activated S-boxes in the application of the 32 S-boxes of Serpent1 during the first and second four iterations is denoted by `getFaultPosition()` which has been explained in Sect. 4.2 and Sect. 4.3. The processes of constructing algebraic equations for SOSEMANUK and faults are respectively denoted by `ConstructEqn()` and `ConstructFaultEqn()`,

which have been all described in Sect.4.4. It is important to note that we only guess the fault positions for the case that the number of non-activated S-boxes during the first four cipher iterations $m \in [2, 3]$. Since in such a case, as mentioned in Sect. 2.1, it is very difficult for the adversary to further prune of the registers candidates where the fault is mostly injected and few equations can be constructed for the fault. Therefore, the adversary should guess the precise fault positions to perform the attack, which implies that the adversary must guess the exact register in which the fault is injected. For the correct value of the guess, the equations that describe the faults will be constructed correctly and the CryptoMiniSat solver will end with “SATISFIABLE” and output a solution within a reasonable time. For the incorrect value of the guess, the CryptoMiniSat solver will end with “UNSATISFIABLE” without any solution output in a very short time.

Algorithm 1: Attack Procedure

Input : the initial K and IV
 $FaultPosition \leftarrow \emptyset$
 $EqnSystem \leftarrow \emptyset$
 $Z \leftarrow \emptyset$
 Run the cipher normally for two cipher iterations with the initial K and IV
 $Z \leftarrow z_8 | z_7 | z_6 | z_5 | z_4 | z_3 | z_2 | z_1$
 $EqnSystem \leftarrow \text{ConstructEqn}(Z)$
for $i = 1$ **to** k **do**
 $FaultPosition \leftarrow \emptyset$
 $Z \leftarrow \emptyset$
 InjectFault (random single-word fault σ_i)
 Run the cipher abnormally for two cipher iterations with K and IV
 $Z^{(i)} \leftarrow z'_8 | z'_7 | z'_6 | z'_5 | z'_4 | z'_3 | z'_2 | z'_1$
 $m \leftarrow$ the number of non-activated S-boxes during the first four iterations
 $n \leftarrow$ the number of non-activated S-boxes during the second four iterations
 if $0 \leq m \leq 31$ **then**
 if $2 \leq m \leq 3$ **then**
 $EqnSystem \leftarrow EqnSystem \cup \text{ConstructEqn}(Z^{(i)})$
 $FaultPosition \leftarrow \text{GuessFaultPosition}(m)$
 end if
 else if $m \geq 10$;
 $EqnSystem \leftarrow EqnSystem \cup \text{ConstructEqn}(Z^{(i)})$
 $FaultPosition \leftarrow \text{getFaultPosition}(m)$
 end if
 if $m \geq 32$ **then**
 $EqnSystem \leftarrow EqnSystem \cup \text{ConstructEqn}(Z^{(i)})$
 $FaultPosition \leftarrow \text{getFaultPosition}(n)$
 end if
 $EqnSystem \leftarrow EqnSystem \cup \text{ConstructFaultEqn}(FaultPosition)$
end
 $\Phi_0 \leftarrow \text{SolveEquationsCryptoMiniSAT}(EqnSystem)$
return Φ_0

6. Theoretical and Simulation Results

This section describes the theoretical and simulation results

and evaluates the complexity of the attack which is mainly composed of the number of fault injections N , time complexity and memory requirement.

Number of Faults Injections N . We use Θ to denote the set of inner states that are deducted from κ . For convenience, we denote by

$$\kappa \xrightarrow{(*)} \Theta \quad (27)$$

the deduction of Θ from κ by equation (*).

We use Ω to denote the set of inner states that are deducted from the fault during the first 8 cipher iterations

$$\Omega = \{s_m, Rn_0 | m, n \in \mathbb{Z}, 1 \leq m \leq 10, 1 \leq n \leq 2\}$$

Then we have ($t \geq 1$):

$$s_{t+10} \xrightarrow{(4)} s_{t+9}, s_{t+3}, s_t \quad (28)$$

$$R1_t \xrightarrow[lsb(R1_0)=1]{(6)} R2_{t-1}, s_{t+1}, s_{t+8} \quad (29)$$

$$R1_t \xrightarrow[lsb(R1_0)=0]{(6)} R2_{t-1}, s_{t+1} \quad (30)$$

$$R2_t \xrightarrow{(7)} R1_{t-1} \quad (31)$$

$$f_t \xrightarrow{(8)} R1_t, R2_t, s_{t+9} \quad (32)$$

From Eq. (28) ~ (32) we have:

$$f_1, f_2, f_3, f_4, f_5, f_6, f_7 \xrightarrow{(4)(6)(7)(8)} \Phi_0 \quad (33)$$

If the adversary injects a fault and the fault has been affected the values of $f_1, f_2, f_3, f_4, f_5, f_6, f_7$, we have:

$$\Omega = \Phi_0 \quad (34)$$

Let k be the number of corrupted f_t by faults during the first eight iterations. Since the fault injections were completely random and the fault positions were uniformly distributed. We have the following results:

1. If $pos \in \{R1_0, R2_0, s_2, s_{10}, s_9 (lsb(R1_0) = 1)\}$, $f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8$ are all corrupted, $P[k = 8] = \frac{9}{24}$
2. If $pos \in \{s_1, s_3, s_4\}$, $f_2, f_3, f_4, f_5, f_6, f_7, f_8$ are all corrupted, $P[k = 7] = \frac{6}{24}$
3. If $pos \in \{s_5\}$, $f_3, f_4, f_5, f_6, f_7, f_8$ are all corrupted, $P[k = 6] = \frac{1}{12}$
4. If $pos \in \{s_6\}$, f_4, f_5, f_6, f_7, f_8 are all corrupted, $P[k = 5] = \frac{1}{12}$
5. If $pos \in \{s_7\}$, f_5, f_6, f_7, f_8 are corrupted, $P[k = 4] = \frac{1}{12}$
6. If $pos \in \{s_8\}$, only f_6, f_7, f_8 are corrupted, $P[k = 3] =$

$$\frac{1}{12}$$

7. If $pos \in \{s_9 (lsb(R1_0) = 0)\}$, only f_7, f_8 are corrupted,
 $P[k = 2] = \frac{9}{24}$

As mentioned in Sect. 3.2, when the induced faults have corrupted $f_1, f_2, f_3, f_4, f_5, f_6, f_7$ during the first eight consecutive cipher iterations, $\Omega = \Phi_0$. Then the probability $P(\Omega = \Phi_0)$ can be calculated as follows:

$$P(\Omega_2 = \Phi_0) = \sum_{i=1}^N \left(1 - (P[k = 8])^{i-1}\right) (P[k = 8])$$

When $P(\Omega_2 = \Phi_0)$ is approximating to 1, $N = 15$. Therefore, the overall expected number of faults is 15.

Time Complexity. The time complexity of the attack refers to the time recovered Φ_0 by solving the combined algebraic equations of SOSEMANUK and faults with the CryptoMinisat 4.4 solver. The time complexity can be calculated as $T = t_1 + (M - 1) \times t_2$, where t_1 is the average solving time of the CryptoMinisat 4.4 solver when the guess is correct, t_2 is the average solving time when the guess is incorrect, and M is the total number of guesses. Since the number of fault injections $N = 15$, $M = 9^{N \times (0.1405 + 0.1218)} \approx 9^4 = 6561$. And experimental results show that when the number of fault injections is 15, $t_1 = 7262.324$ seconds, $t_2 = 1.07$ seconds. Thus, we have $T = t_1 + 6560 \times t_2 \approx 3.97$ hours.

Memory requirement. The memory requirement of the attack mainly depends on the solver. For each attack instance the solver will automatically output the memory cost. And since the CryptoMiniSat 4.4 solver only cares about how to find out a solution for all the variables (including both known and unknown variables) which satisfies all the equations, the memory cost is not only depended on the bound of m that is guessed, but also the searching strategy of the solver. Thus, it is very hard to evaluate the memory requirement like [18] and [19]. The memory amount required for our attack can only be roughly estimated which is equal to $2^{38.17}$ bytes.

Simulation Results of the attack. In order to verify the effectiveness of the proposed attack on SOSEMANUK, we conduct many experiments and report the results in this section. We conduct the fault injection with software level simulations. The CryptoMiniSat 4.4 solver is running on a PC with Intel Core i7-4790, 3.60 GHZ, 12G memory, Windows 7 64-bit OS. An instance refers to one run of our attack on a set of K, IV . The instance fails if the solver does not give an output within 48 hours or end with “UNSATISFIABLE”.

To calculate the success rate of the proposed method of determining fault positions and to evaluate the number of fault injections, 10,000 instances are tested with different plaintexts and keys. For each instance, we first use a secret key K and IV to generate the initial inner state Φ_0 and get the correct keystream Z . Then, we inject random single-word faults right after the SOSEMANUK initialization step within the inner state to get the faulty ciphertext Z' . Finally, we use the method in Sect. 4.3 to build the algebraic equations for

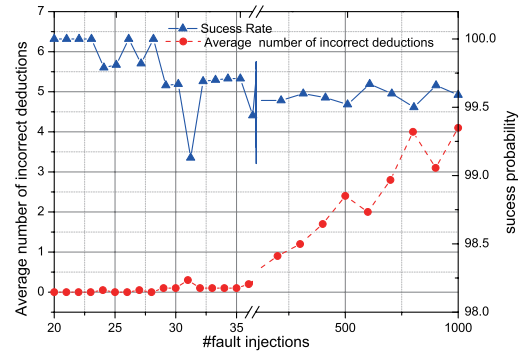


Fig. 3 The average number of incorrect deductions on the fault positions and the success probability in determining fault position vs the number of fault injections.

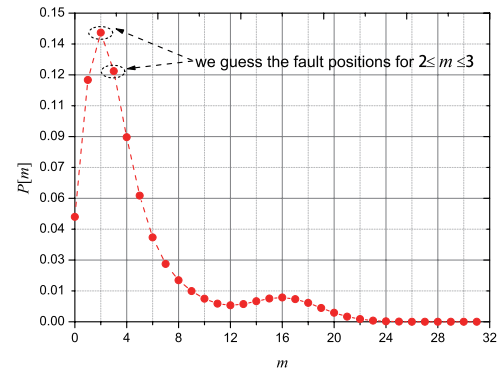


Fig. 4 The probability $P[m]$ vs the number of non-activated S-boxes during the first four cipher iterations m .

the cipher and the faults, and solve them with CryptoMiniSat 4.4 to deduce the the initial inner state Φ_0 .

Figure 3 shows the statistics of the average number of incorrect deductions on the fault positions and success rate versus the number of fault injections. We can see that the success rate of the proposed method is greater than 99%.

Figure 4 shows the probability $P[m]$ versus the number of non-activated S-boxes during the first four cipher iterations m . It can be seen that $P[m = 2] = 0.1405$ and $P[m = 3] = 0.1218$ are the two maximum values. Thus, in the **GuessFaultPosition** procedure of the attack, we guess fault positions for $m \in [2, 3]$. For comparison, we also conduct experiments on guessing fault positions for $m \in [0, 9]$.

Figure 5 shows the statistics of the average solving time of CryptoMinisat4.4 solver versus the number of fault injections. It can be seen that the number of fault injections to obtain the 384 bits Φ_0 are all 15 for $m \in [0, 9]$ and $m \in [2, 3]$. Thus indicating the simulation results are consistent to the theoretical ones where the number of fault injections is $N = 15$.

The figure also clearly shows the solving time corresponding to $m \in [2, 3]$ for the correct guess (7262.324 seconds on average when the number of fault injections is 15) having a much higher average compared to $m \in [0, 9]$

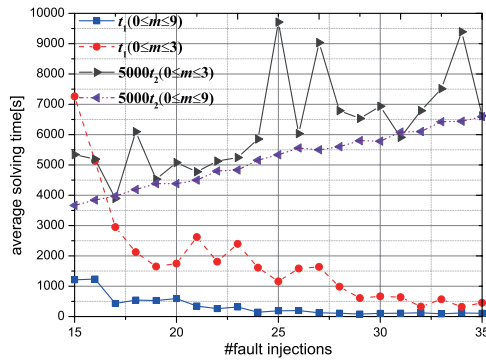


Fig. 5 The average solving time vs the number of fault injections.

(1209.660 seconds on average), while the average solving time corresponding to $m \in [2, 3]$ (1.071 seconds on average) for the incorrect guess is very close to the value corresponding to $m \in [0, 9]$ (0.772s on average). However, the time complexity of the attack corresponding to $m \in [0, 9]$ (5251700 hours) is far larger than the value corresponding to $m \in [2, 3]$ (3.97h) due to the total number of guesses for $m \in [0, 9]$ ($8^{15 \times 0.1685} \times 9^{15 \times 0.4544} \times 10^{15 \times 0.0632} \approx 8^3 \times 9^7 \times 10$) is much larger than the value for $m \in [2, 3]$ (6561).

7. Conclusions

In this paper, we have proposed an improved fault attack of SOSEMANUK by using algebraic techniques base on a single-word fault model, which only needs 15 fault injections to recover the whole initial inner states of SOSEMANUK within 3.97 hours. The results of simulations on a PC showed that the number of fault injections required in the simulation is consistent to the estimated values. The proposed attack in this paper is based on the idea that the ARX operations as well as the injected faults can be represented as a series of algebraic equations and the inner state can be recovered by solving the combined equations with a SAT solver. our attack can be extended to other ciphers using additions modulo 2^n , multiplications by a primitive element, S-boxes, bit rotations and XORs, which will be left as future work.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under the grants 61272491, 61309021, 61472357, 61571063, by the Major State Basic Development Program (973 Plan) 2013CB338004, by the Fundamental Research Funds for the Central Universities under the grant 2015QNA5005 and by the Science and Technology on Communication Security Laboratory under the grant 9140C110602150C11053.

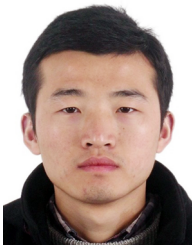
References

- [1] D. Boneh, R.A. DeMillo, and R.J. Lipton, "On the importance of checking cryptographic protocols for faults," EUROCRYPT 1997,

- LNCS, vol.1233, pp.37–51, 1997.
- [2] E. Biham and A. Shamir, "Differential fault analysis of secret key cryptosystems," CRYPTO 1997, LNCS, vol.1294, pp.513–525, 1997.
- [3] J. Hoch and A. Shamir, "Fault analysis of stream ciphers," CHES 2004, LNCS, vol.3156, pp.240–253, 2004.
- [4] I. Biehl, B. Mwyer, and V. Muller, "Differential fault analysis on elliptic curve cryptosystems," CRYPTO 2000, LNCS, vol.1880, pp.131–146, 2000.
- [5] J. Blomer and J.-P. Seifert, "Fault based cryptanalysis of the advanced encryption standard (AES)," Financial Cryptography and network security-FC 2003, LNCS, vol.2742, pp.162–181, 2003.
- [6] G. Piret and J. Quisquater, "A differential fault attack technique against SPN structures, with application to the AES and Khazad," CHES 2003, LNCS, vol.2779, pp.77–88, 2003.
- [7] W. Li, D. W, and J.R. Li, "Differential fault analysis on the ARIA algorithm," Inform. Sciences, vol.178, no.19, pp.3727–3737, 2008.
- [8] J. Takahashi, T. Fukunaga, and K. Sakiyama, "Differential fault analysis on stream cipher MUGI," IEICE Trans. Fundamentals, vol.E95-A, no.1, pp.242–251, 2012.
- [9] M. Soos, K. Nohl, and C. Castelluccia, "Extending SAT solvers to cryptographic problems," SAT 2009, LNCS, vol.5584, pp.244–257, 2009.
- [10] M. Hojsik and B. Rudolf, "Differential fault analysis of Trivium," FSE 2008, LNCS, vol.5086, pp.58–172, 2008.
- [11] M. Mohamed, S. Bulygin, and J. Buchmann, "Using SAT solving to improve differential fault analysis of trivium," ISA 2011, CCIS 200, pp.62–71, 2011.
- [12] P. Jovanovic, M. Kreuzer, and I. Polian, "A fault attack on the LED block cipher," COSADE 2012, LNCS, vol.7275, pp.120–134, 2012.
- [13] M. Mohamed, S. Bulygin, M. Zohner, A. Heuser, and M. Walter, "Improved algebraic side-channel attack on AES," Cryptology Print Archive. <http://eprint.iacr.org/2012/084.pdf>, 2012.
- [14] M. Renauld and F.-X. Standaert, "Algebraic side-channel attacks," INSCRYPT 2009, LNCS, vol.6151, pp.393–410, 2009.
- [15] S. Banik and S. Maitra, "Improved fault attack on MICKEY 2.0," J. Cryptographic Engineering, vol.5, no.1, pp.215–232, 2013.
- [16] X. Zhao, S. Guo, F. Zhang, Z. Shi, C. Ma, and T. Wang, "Improving and evaluating differential fault analysis on LED with algebraic techniques," Proc. FDTC 2013, pp.41–51, 2013.
- [17] N. Courtois and J. Pieprzyk, "Cryptanalysis of block ciphers with overdefined system of equations," ASIACRYPT 2002, LNCS, vol.2501, pp.267–287, 2002.
- [18] Y.E. Salehani, A. Kircanski, and A. Youssef, "Differential fault analysis of sosemanuk," AFRICACRYPT 2011, LNCS, vol.6737, pp.316–331, 2011.
- [19] Z. Ma and D. Gu, "Improved differential fault analysis of SOSEMANUK," 2012 Eighth International Conference on Computational Intelligence and Security, pp.487–491, 2012.
- [20] C. Berbain, O. Billet, A. Canteaut, N. Courtois, H. Gilbert, L. Goubin, A. Gouget, L. Granboulan, C. Lauradoux, M. Minier, T. Pornin, and H. Sibert, "SOSEMANUK, a fast software-oriented stream cipher," eSTREAM, the ECRYPT Stream Cipher Project, Report 2005/027, 2005.
- [21] The eSTREAM project, <http://www.ecrypt.eu.org/stream/>
- [22] P. Ekdahl and T. Johansson, "A new version of the stream cipher SNOW," In: W. Kuich, G. Rozenberg, and A. Salomaa eds., DLT 2001. LNCS, vol.2295, pp.47–61. Springer, Heidelberg, 2002.
- [23] B. Gladman, "SERPENT performance," <http://fp.gladman.plus.com/cryptographytechnology/serpent/>
- [24] P. Ekdahl and T. Johansson, "SNOW-a new stream cipher," <http://www.cyptonessie.org>, 2000.

Table A · 1 The full result for determining the fault positions of all cases.

m	$\Pr[C_{i=5}[6 7]]$	$\Pr[C_{i=4}]$	$\Pr[C_{i=3}]$	$\Pr[C_{i=2}]$	$\Pr[C_{i=1}]$	$\Pr[\kappa]$	κ
0	0	3.8015e-010	1.6401e-004	0.0683	0.9316	0.9999	$\{s_1, s_2, s_3, s_4, s_9, s_{10}, R_{10}, R_{20}\}$
1	0	5.2863e-009	7.6024e-004	0.1356	0.8636	0.9992	
2	0	6.8472e-008	0.0033	0.2510	0.7457	1	
3	0	7.9007e-007	0.0126	0.4137	0.5736	0.9999	$\{s_1, s_2, s_3, s_4, s_5, s_9, s_{10}, R_{10}, R_{20}\}$
4	0	7.7797e-006	0.0414	0.5820	0.3766	1	
5	0	6.3734e-005	0.1132	0.6811	0.2057	1	
6	0	4.2833e-004	0.2535	0.6539	0.0921	0.9995	$\{s_1, s_2, s_3, s_4, s_5, s_6, s_9, s_{10}, R_{10}, R_{20}\}$
7	0	0.0023	0.4580	0.5064	0.0333	1	
8	0	0.0101	0.6651	0.3151	0.0097	1	
9	0	0.0365	0.7990	0.1622	0.0023	1	$\{s_1, s_3, s_4, s_5, s_6\}$
10	0	0.1118	0.8167	0.0711	4.7499e-004	0.9996	
11	0	0.2836	0.6906	0.0258	8.0333e-005	1	
12	0	0.5480	0.4449	0.0071	1.0350e-005	1	$\{s_5, s_6\}$
13	0	0.7859	0.2127	0.0015	9.8951e-007	1	
14	0	0.9170	0.0827	2.4286e-004	7.6977e-008	0.9997	
15	0	0.9708	0.0292	3.6727e-005	5.4325e-009	1	$\{s_5\}$
16	0	0.9901	0.0099	5.3510e-006	3.6937e-010	1	
17	0	0.9967	0.0033	7.6953e-007	2.4789e-011	1	
18	0	0.9989	0.0011	1.1018e-007	1.6563e-012	1	$\{s_7, s_8, s_9\}$
19	0	0.9996	3.7107e-004	1.5751e-008	1.1050e-013	1	
20	0	0.9999	1.2372e-004	2.2507e-009	7.3684e-015	1	
21~31	0	1	0	0	0	1	
32	1	0	0	0	0	1	



Hao Chen was born in 1987. He received his M.S. degree in the Department of Information Engineering, Ordnance Engineering College, Shijiazhuang, China. He is currently a Ph.D. candidate in the same Department. His main research interest is fault analysis of stream ciphers.



Xinjie Zhao received his M.S. and Ph.D. degrees in the Department of Information Engineering, Ordnance Engineering College, Shijiazhuang in 2012 and 2009, respectively. He is currently an Engineer in Institute of North Electronic Equipment. His main interests include side channel analysis, fault analysis of block ciphers. He won the best paper award in Darmstadt-the 3rd International Workshop on Constructive Side-Channel Analysis and Secure Design (COSADE 2012).



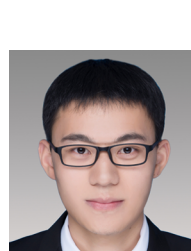
Tao Wang received his Ph.D. degree in computer application from Institute of Computing Technology Chinese Academy of Sciences in 1966 and master's degree in computer application from Ordnance Engineering College in 1990. He is currently a professor in Ordnance Engineering College. His research interests include information security and cryptography.



Fan Zhang is working in the College of Information Science & Electrical Engineering, Zhejiang University, China. He received his Ph.D. degree in the Department of Computer Science and Engineering from University of Connecticut in 2012. His main interests include side channel analysis and fault analysis in cryptography, computer architecture, and security in wireless network.



Shize Guo was born in 1964. He received his Ph.D. degree in Harbin Institute of Technology in 1989 and his M.S. and B.S. degrees from Ordnance Engineering College, China, in 1991 and 1988, respectively. He is currently a researcher in Institute of North Electronic Equipment and also a Professor in Beijing University of Post and Telecommunications. His main research interest includes information technology and information security.



Jian Liu was born in 1990. Currently he is a Ph.D. candidate in College of Communication and Information Engineering at Nanjing University of Posts and Telecommunications (NJUPT), Nanjing, China. His research interests are in the areas of stochastic resonance (SR) and its applications, including signal detection, signal transmission, and digital communication system.